

INPAINTING

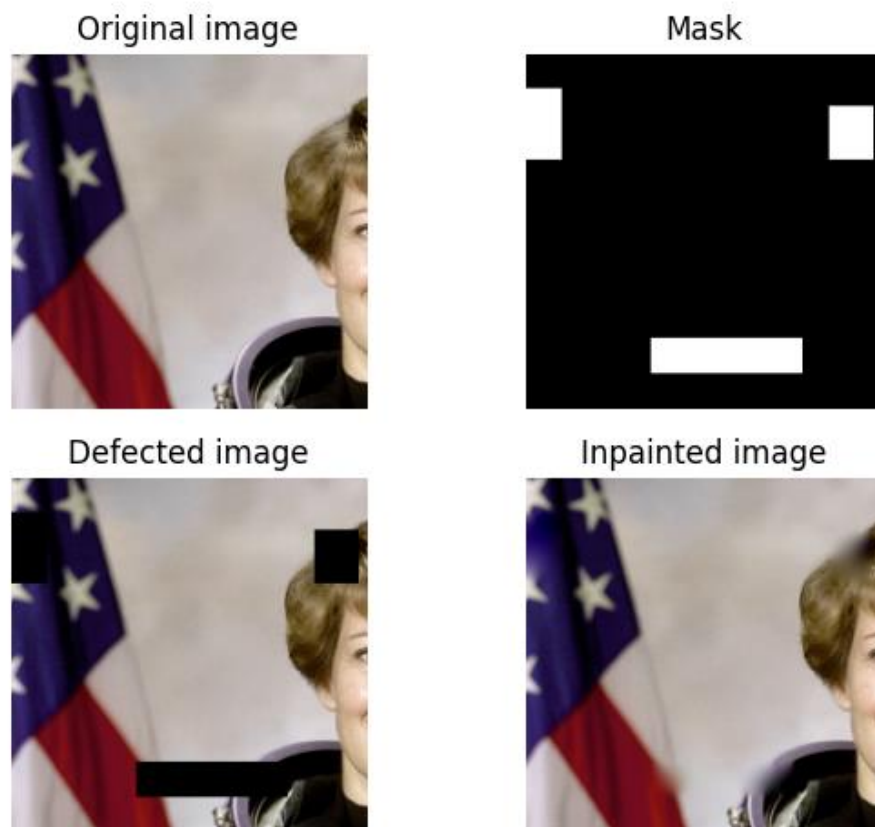
ENTREGA: Importante, leer la parte final de este documento en las cuales se describen las normas de entrega de la practica

The aim of this practical session is to implement an inpainting algorithm.

To start, please have a look to the following example provided in the skimage library:

https://scikit-image.org/docs/stable/auto_examples/filters/plot_inpaint.html

It is important to note that the result shown in the image below is the optimal goal. However, in this practical session we will try to approximate the results with the inpainting steps described below.



Use the following code to load the image

```
import numpy as np
```

```

import matplotlib.pyplot as plt
from skimage import data
mask_rgb=[0,0,0]
image_orig = data.astronaut()[0:200,0:200]
pad_size = 60
# Create mask with three defect regions: left, middle, right respectively
mask = np.zeros(image_orig.shape[:-1])
mask[20:60, 0:20] = 1
mask[160:180, 70:155] = 1
mask[30:60, 170:195] = 1

# Defect image over the same region in each color channel
image_defect = image_orig.copy()
for layer in range(image_defect.shape[-1]):
    image_defect[np.where(mask)] = mask_rgb

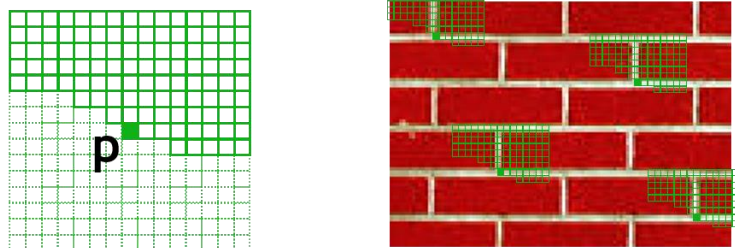
subplot, arr = plt.subplots(1, 1)
arr.imshow(image_defect, cmap='gray')
arr.set_title("Image with defects")
plt.gcf().set_size_inches((6,6))

```

As it can be seen in the figure, from an original image (top left), a few regions of a mask are removed (top right). The defected image (bottom left) is used as input of the algorithm and a recovered / inpainted image (bottom right) is obtained.

The steps of the practical session are the following

- 1) Load the astronaut sample image and remove the mask, similarly to the example https://scikit-image.org/docs/stable/auto_examples/filters/plot_inpaint.html
- 2) Pattern search
 - a. For each missing pixel search the input image for all similar neighborhoods using the SSD (Sum of Square Distances), Use a patch 11x11 pixel wide

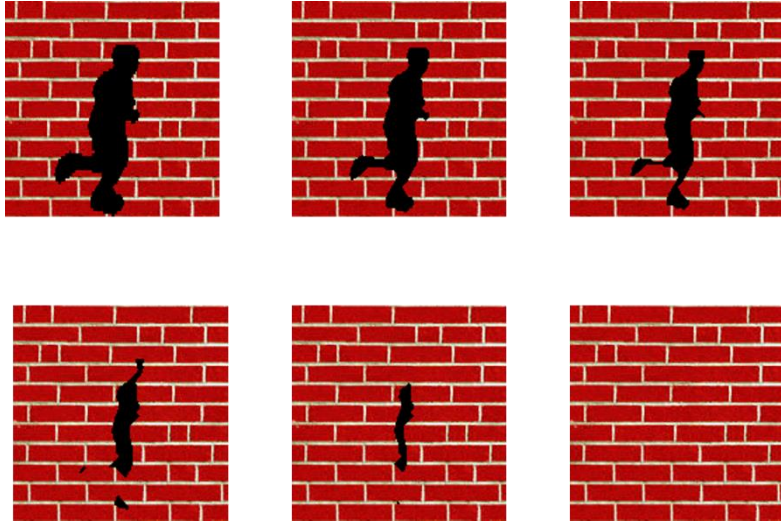


- b. To speed up the computation instead of searching the whole image, only search in a 21x21 area around the pixel “p” (i.e. 5 pixel on each side more than the 11x11 patch)

3) Filling strategy

- a. Fill the image following the “Onion Ring” strategy (pixels with most neighbors are synthesized first).

Suggestion: the pixel of the mask (Onion Ring) can be obtained using an “erosion” technique (see the binary image processing theory and live code).



- b. (Facultative) To obtain a better final mark of the practical session, the student can implement the Gradient-based filling instead of the onion ring strategy

Please submit all the material in the campus virtual **before the deadline**.

- Upload all the python notebook and the images used in a single jupyter notebook file.
- Comment the code, discuss the problem, limitations and improvements.
- Please note that optional improvement will increase the final mark (see 3B below). For obtaining a sufficient >5 mark, the steps until 3A included are mandatories. They should be correctly implemented and the code commented.